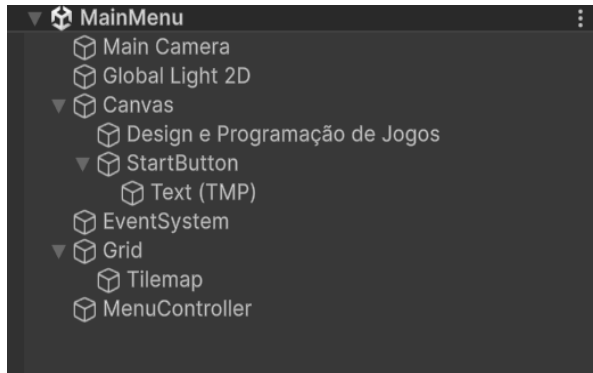


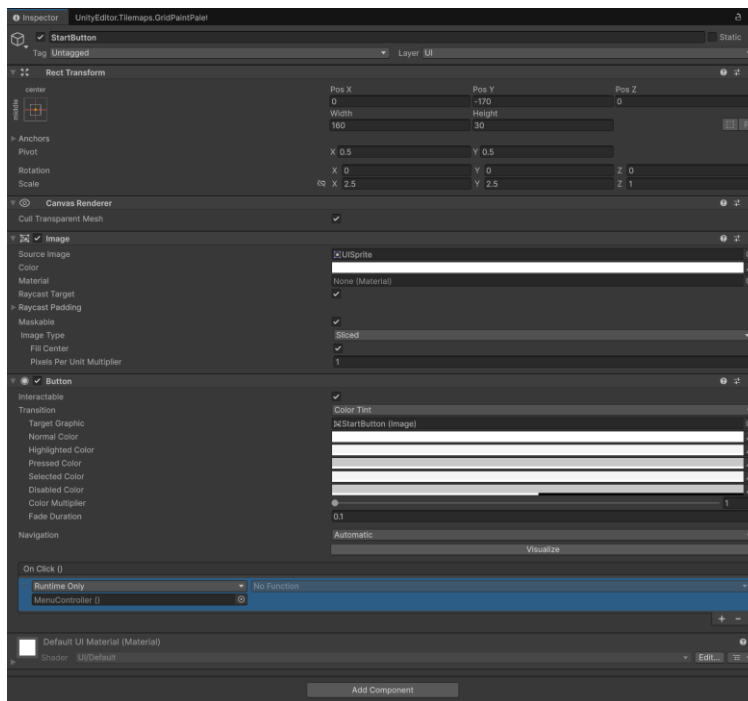
Technical Document Lab 1

Design e Programação de Jogos (DPJ)

Level 1

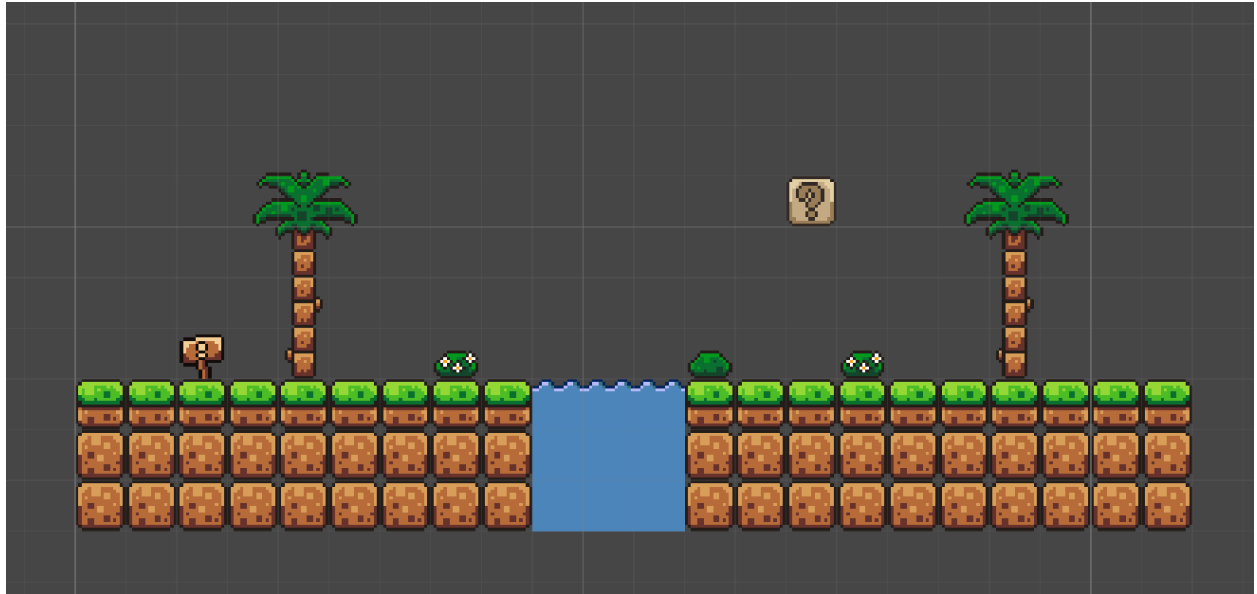


This is all the gameObjects that create the scene MainMenu. Inside it has a canvas with the title of the game, a start button with a text, and behind the canvas, there is a tilemap that creates the scenery of a 2D platformer.



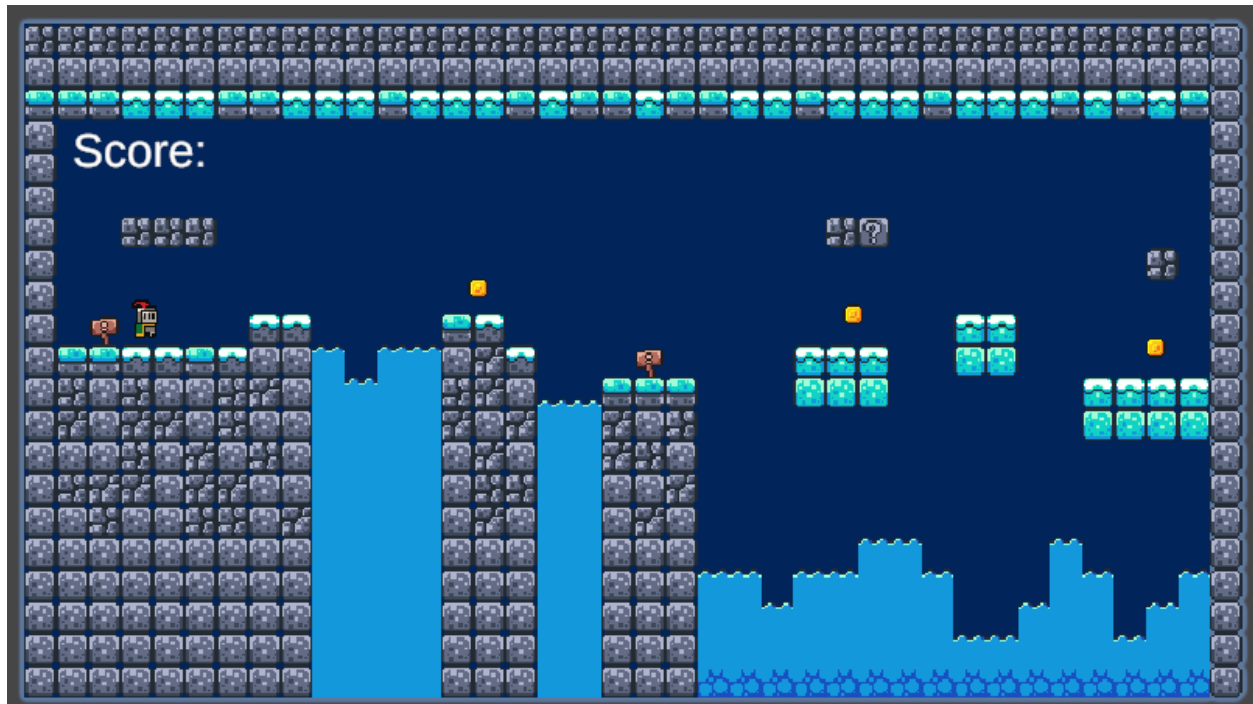
This is the inspector for the gameObject "StartButton". On the top of the inspector, you can adjust the height and length of the button or lock it in a specific zone. In the middle of the inspector, in the Image section, you can change the font, the color of the background, the font size, and a section where you can add an image for the background or for the button altogether. Finally, on the bottom you can check what the button does. When you click the button, it will activate a code called "MenuController" and

activate the game.



This is the Main Menu background, it was made with a tilemap, and it consists of a 2D map, with a few trees, a gap with water, and a few decorations around it. It's a scene that will welcome the player into the game before starting to play it.

Level 2



This is the second level that we made, it consists of an underground level with a few jumps from pillar to pillar, and a few platform jumps that lead to bodies of water and It also has some decorations.

```
using UnityEngine;

Script de Unity | 0 referências
public class Collectible : MonoBehaviour
{
    public int scoreValue = 1;
    public bool destroyOnCollect = true;

    Mensagem do Unity | 0 referências
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            if (GameManager.Instance != null)
            {
                GameManager.Instance.AddScore(scoreValue);
            }
            else
            {
                Debug.LogWarning("GameManager não encontrado na cena. Certificar que existe um GameManager com script anexado.");
            }

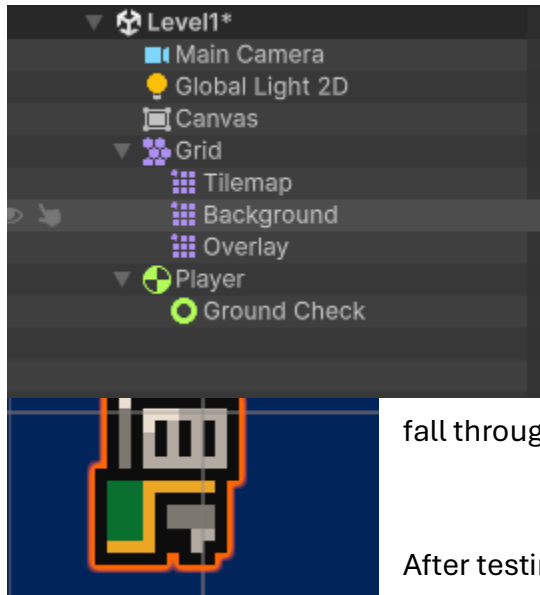
            if (destroyOnCollect)
            {
                Destroy(gameObject);
            }
            else
            {
                gameObject.SetActive(false);
            }
        }
    }
}
```

The collectible object acts as a pickup item for the player in the game. When an object tagged "Player" touches its trigger collider, the script checks for the GameManager to add points (scoreValue) to the player's score. If no GameManager is found in the scene, it throws a warning in the console. Finally, based on the destroyOnCollect setting, the item is either permanently destroyed from memory or simply hidden from the scene (SetActive(false)).



Shown in the picture are the collectible's Box Collider, model and Script which are the fundamental for the game, which will allow the player to collect it for more points. Each coin gives the player 1 point, and the player has to collect all of them to get 3 points.

Level 3



This shows the hierarchy of level 1, which has 3 different tilemaps, one for the map itself, the other for the background, and the final tilemap is for the special details of the map.

This is the character that the player will control, it has a ground check as a child object that checks if the player is touching the ground. Or else he'll fall through the objects and disappear.

After testing the different Rigidbody types on both the character and the environment, and studying their behaviors,

we learned the following:

- **Dynamic:** It's fully driven by the physics engine. It responds to forces, friction, and gravity, and collides with all other body types. Great for playable characters and dynamic physical objects.
- **Kinematic:** It's controlled strictly via script or direct transform movement, completely ignoring gravity and physical forces. It only collides with *Dynamic* bodies and passes through other Kinematic or Static bodies. Ideal for moving platforms, elevators, or scripted disasters.
- **Static:** It's mostly designed to remain completely stationary. It has infinite mass, ignores all forces and scripts, and acts as a solid physical barrier. It's useful for environmental elements like floors, walls, and ground tilemaps.

Unity gives us three different Rigidbody 2D body types to control physical behavior, which is really grate because it'll let us choose the exact physics logic needed for each object, optimizing both game performance and control.

By using Dynamic for the player, we ensure natural collisions and movement physics, using Kinematic for interactive obstacles or moving platforms gives us precise, script-driven movement without unwanted interference, and applying Static to the environment prevents unnecessary physics calculations on fixed objects, keeping the game running smoothly and keeping it fun.

Level 4



All of the components we added in this level, consisting of three respawn triggers with a script to respawn when touched, 3 collectible coins, a game object that controls the Score text and a respawn point for the player if he falls on one of the triggers.



As shown in the picture, when the game starts, the score starts at zero.



And after collecting the coin the game registers a point to the score, making the coin disappear

In Conclusion

In Level 1, we studied the main unity Components such as the inspector, the hierarchy, the gameObjects inside it and detailed them.

In Level 2, we made an underground platformer map with Tilemap that has pillar jumps, static structures, and water that kills the player when he dies. We created a collectible coin prefab using a SpriteRenderer, a BoxCollider2D set as a trigger, and the Collectible.cs script. When the player contacts a coin, the script communicates with the GameManager to award a point and destroys or hides the coin object.

In Level 3, we set up the Player GameObject using a SpriteRenderer, Rigidbody2D, BoxCollider2D, and the script PlayerController.cs. We placed an empty child GameObject called Ground Check at the player's feet to detect when the player is touching the ground. We structured the map across three Tilemap layers for the main ground, background, and overlay details, and documented the three Rigidbody 2D types: Dynamic for physics-driven player movement, Kinematic for script-controlled objects that ignore gravity, and Static for stationary obstacles like ground tiles.

In Level 4, we added a GameManager connected to a TextMeshPro UI that tracks collected coins in real time, starting from zero and increasing as coins are picked up. We placed Respawn Triggers inside the water gaps so that touching them instantly teleports the player back to a designated Respawn Point at the start of the level.

We completed the assignment given and learned a lot on how to make a 2D game, we learned physics, colliders, how to make a map using Tilemap, how to make collectible objects, and how to count the score.

We really liked how it turned out and we're grateful that we got this opportunity to learn.